

Obrada transakcija i ACID izolacija u PostgreSQL-u

Seminarski rad — Sistemi za upravljanje bazama podataka

Agenda

01 Šta je transakcija?

Osnove i životni ciklus

02 ACID svojstva

Atomičnost, Konzistentnost, Izolacija, Trajnost

03 Fenomeni konkurentnog izvršavanja

Dirty Read, Non-Repeatable, Phantom Read

04 Nivoi izolacije

Read Committed, Repeatable Read, Serializable

05 MVCC mehanizam

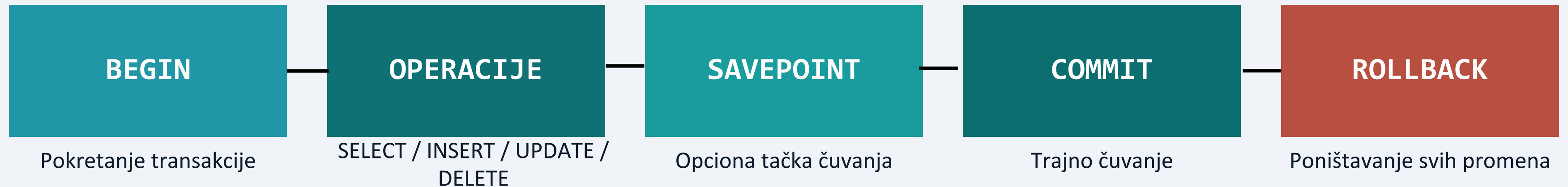
Kako PostgreSQL interno funkcioniše

06 Praktični primeri

12 demonstracija u pgAdmin-u

Šta je transakcija?

Logička jedinica rada — sve operacije uspevaju zajedno ili nijedna.



```
BEGIN;  
  UPDATE racuni SET stanje = stanje - 1000 WHERE id = 1;  
  SAVEPOINT tacka1;  
  UPDATE racuni SET stanje = stanje + 1000 WHERE id = 2;  
COMMIT; - ili ROLLBACK TO SAVEPOINT tacka1;
```

Primer: transfer novca između dva računa

ACID svojstva

A Atomičnost

Sve ili ništa — nema delimičnih transakcija

PostgreSQL:

Write-Ahead Log (WAL)

C Konzistentnost

Validno stanje podataka pre i posle

PostgreSQL:

Constraints + Triggers

I Izolacija

Transakcije se ne ometaju međusobno

PostgreSQL:

MVCC mehanizam

D Trajnost

Commit je permanentan čak i pri padu

PostgreSQL:

WAL + fsync +
Checkpoint

Fenomeni konkurentnog izvršavanja

Dirty Read

Prljavo čitanje

T2 čita uncommitted podatke od T1. Ako T1 uradi ROLLBACK, T2 je radila sa nepostojećim podacima.

Sprečava:
Read Committed

Non-Repeatable Read

Neponovljivo čitanje

Isti SELECT vraća različite vrednosti jer je druga transakcija izmenila i potvrdila red između dva upita.

Sprečava:
Repeatable Read

Phantom Read

Fantomsko čitanje

Isti upit vraća različit broj redova — druga transakcija je ubacila ili obrisala redove koji odgovaraju uslovu.

Sprečava:
Serializable (+ Rep. Read u PG)

Serialization Anomaly

Serializaciona anomalija

Rezultat paralelnih transakcija nije ekvivalentan nijednom mogućem serijskom redosledu izvršavanja.

Sprečava:
Serializable

Nivoi izolacije u PostgreSQL-u

Nivo izolacije	Prljavo čitanje	Neponovljivo čitanje	Fantomsko čitanje	Serijalizaciona anomalija
Read Uncommitted*	Sprečeno*	Moguće	Moguće	Moguće
Read Committed	Sprečeno	Moguće	Moguće	Moguće
Repeatable Read	Sprečeno	Sprečeno	Sprečeno*	Moguće
Serializable	Sprečeno	Sprečeno	Sprečeno	Sprečeno

* PostgreSQL tretira Read Uncommitted kao Read Committed. Repeatable Read u PG-u sprečava i fantomska čitanja.

MVCC — Multi-Version Concurrency Control

Čitanje ne blokira pisanje. Pisanje ne blokira čitanje.

Transakcija T1
čita: stanje = 1.000 din

xmin=100 xmax=200
stanje = 1.000 din (stara verzija)

xmin=200 xmax=0
stanje = 2.000 din (nova verzija)

Transakcija T2
piše: stanje = 2.000 din

- Prikaz MVCC sistemskih kolona

```
SELECT xmin, xmax, ctid, id, vlasnik, stanje  
FROM racuni;
```
- xmin = ID transakcije koja je kreirala verziju
- xmax = ID transakcije koja je obrisala/ažurirala (0 = aktivan red)

VACUUM automatski uklanja stare verzije koje nijedna transakcija više ne vidi.

Praktični primeri — Priprema testnog okruženja

Bankarski sistem: 4 tabele, 3 korisnika, 3 artikla

```
CREATE TABLE racuni (  
  id SERIAL PRIMARY KEY,  
  vlasnik VARCHAR(100) NOT NULL,  
  stanje NUMERIC(12,2) NOT NULL,  
  CONSTRAINT pozitivno_stanje CHECK (stanje >= 0) -- konzistentnost!  
);  
  
INSERT INTO racuni (vlasnik, stanje) VALUES  
  ('Ana', 10000.00), ('Bojan', 5000.00), ('Carla', 2000.00);
```

Ana: 10.000 din

Bojan: 5.000 din

Carla: 2.000 din

Primer 1 — COMMIT i ROLLBACK (Atomičnost)

Scenarij: prenos 1.000 din od Ane ka Bojanu

✓ COMMIT

```
BEGIN;  
  UPDATE racuni SET stanje = stanje - 1000 WHERE  
id = 1;  - Ana  
  UPDATE racuni SET stanje = stanje + 1000 WHERE  
id = 2;  - Bojan  
  SELECT id, vlasnik, stanje FROM racuni;  
COMMIT;
```

- Rezultat: Ana = 9.000, Bojan = 6.000

✗ ROLLBACK

```
BEGIN;  
  UPDATE racuni SET stanje = stanje - 5000 WHERE  
id = 1;  
  UPDATE racuni SET stanje = stanje + 5000 WHERE  
id = 2;  
ROLLBACK;
```

- Rezultat: stanja ostaju nepromenjena!
- Ana = 9.000, Bojan = 6.000 (kao pre)

Ključna poruka: transakcija koja nije potvrđena ne ostavlja nikakav trag u bazi podataka.

Primer 2 — Dirty Read (Izolacija)

Tab 1 menja podatak ali NE potvrđuje. Tab 2 pokušava da čita.

Tab 1 — Sesija 1

```
BEGIN;  
UPDATE racuni  
  SET stanje = 99999  
  WHERE id = 1;  
- JOŠ NIJE COMMIT!
```

Tab 2 — Sesija 2

```
SELECT id, vlasnik, stanje  
FROM racuni WHERE id = 1;  
  
- Rezultat: Ana | 9.000  
- (staro, potvrđeno stanje!)
```

Zaključak: PostgreSQL na nivou Read Committed (podrazumevanom) NIKADA ne dozvoljava čitanje nepotvrđenih podataka. Čak ni na Read Uncommitted nivou — PostgreSQL ga tretira identično kao Read Committed.

Primer 3 — Non-Repeatable Read vs Repeatable Read

Read Committed — fenomen moguć

```
- Tab 1 (Read Committed)
BEGIN;
SELECT stanje FROM racuni
  WHERE id = 2;
- Rezultat: 6.000

- Tab 2: UPDATE stanje=7000; COMMIT;

SELECT stanje FROM racuni
  WHERE id = 2;
- Rezultat: 7.000 ← RAZLIČITO!
```

Repeatable Read — zaštita aktivna

```
- Tab 1 (Repeatable Read)
BEGIN ISOLATION LEVEL
  REPEATABLE READ;
SELECT stanje FROM racuni
  WHERE id = 2;
- Rezultat: 5.000

- Tab 2: UPDATE stanje=8000; COMMIT;

SELECT stanje FROM racuni
  WHERE id = 2;
- Rezultat: 5.000 ← ISTO!
```

Primer 4 — Phantom Read i zaštita

Tab 1 broji redove; Tab 2 ubacuje novi; Tab 1 ponovo broji

Read Committed — fantom se pojavljuje

```
BEGIN;  
SELECT COUNT(*) AS broj  
FROM artikli;  
- Rezultat: 3  
  
- Tab 2 ubacuje 'Tastatura'; COMMIT  
  
SELECT COUNT(*) AS broj  
FROM artikli;  
- Rezultat: 4 ← FANTOM!
```

Repeatable Read — PostgreSQL blokira fantom

```
BEGIN ISOLATION LEVEL  
    REPEATABLE READ;  
SELECT COUNT(*) AS broj  
FROM artikli;  
- Rezultat: 3  
  
- Tab 2 ubacuje 'Slušalice'; COMMIT  
  
SELECT COUNT(*) AS broj  
FROM artikli;  
- Rezultat: 3 ← ZAŠTIĆENO!
```

PostgreSQL je strožiji od SQL standarda — Repeatable Read sprečava i fantomska čitanja zahvaljujući MVCC arhitekturi.

Primer 5 — SELECT FOR UPDATE i NOWAIT

Tab 1 zaključava red; Tab 2 čeka ili odmah dobija grešku

FOR UPDATE — čekanje

- Tab 1:
BEGIN;
SELECT * FROM racuni
WHERE id = 1 FOR UPDATE;
- Red je zaključan!
- Tab 2 čeka...
UPDATE racuni SET stanje = stanje - 100
WHERE id = 1;
- Waiting for the query to complete...
- (čekao 54 sekunde!)

FOR UPDATE NOWAIT — odmah greška

- Tab 2 (dok Tab 1 drži lock):
BEGIN;
SELECT * FROM racuni
WHERE id = 1
FOR UPDATE NOWAIT;
- ERROR: could not obtain lock
- on row in relation "racuni"
- SQL state: 55P03
- Aplikacija reaguje odmah!

Koristi NOWAIT kada aplikacija ne može da čeka i mora odmah da reaguje na nedostupnost resursa.

Primer 6 — Deadlock detekcija

- Tab 1:

BEGIN;

UPDATE racuni SET stanje =
 stanje - 100 WHERE id = 1;

- zaključava red 1 ✓

UPDATE racuni SET stanje =
 stanje + 100 WHERE id = 2;

- čeka na red 2...

- Tab 2 (paralelno):

BEGIN;

UPDATE racuni SET stanje =
 stanje - 100 WHERE id = 2;

- zaključava red 2 ✓

UPDATE racuni SET stanje =
 stanje + 100 WHERE id = 1;

- čeka na red 1...

- DEADLOCK!

ERROR: deadlock detected

Process 13180 waits for ShareLock on transaction 830; blocked by process 21960.

Process 21960 waits for ShareLock on transaction 831; blocked by process 13180.

PostgreSQL automatski detektuje deadlock i poništava jednu od transakcija. Rešenje: uvek pristupati resursima u istom redosledu.

Primer 7 — Dijagnostika blokiranih transakcija

Alat za praćenje blokiranih transakcija u realnom vremenu

```
SELECT
  blocked.pid          AS blokirani_pid,
  blocking.pid         AS blokator_pid,
  blocked.query        AS blokirani_upit,
  blocking.query       AS blokator_upit
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
  ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE blocked.wait_event_type = 'Lock';
```

blokirani_pid	blokator_pid	blokirani_upit
13180	21960	BEGIN;

Administrator odmah vidi koji proces blokira koji — i može prekinuti problematičnu transakciju.

Zaključak

✓ ACID garantuje pouzdanost

Atomičnost, konzistentnost, izolacija i trajnost su temelj svake transakcione baze podataka.

✓ MVCC = konkurentnost bez blokiranja

Čitanje i pisanje mogu se odvijati istovremeno zahvaljujući višestrukim verzijama redova.

✓ PostgreSQL je strožiji od standarda

Repeatable Read sprečava i fantomska čitanja — bolje od minimalnih zahteva SQL standarda.

✓ Izbor nivoa izolacije je kompromis

Read Committed za OLTP, Repeatable Read za izveštaje, Serializable za kritične transakcije.

Hvala na pažnji!